Gödel's Second Incompleteness Theorem in a Guard/Rose Equational System: Formalization in Agda

1 Overview

We formalize Gödel's first and second incompleteness theorems in a purely equational system combining two ideas:

- *Guard*'s syntax [1]: binary trees with typed unary and binary functors, serving as both the data domain and the term language;
- *Rose*'s purely equational proof style [2]: the only judgement is an equation $t = u$ between terms, with no propositional connectives, no sequents, and no logical rules — only computation axioms, congruence, substitution, and a tree-induction principle (**Schema F**).

This combination is a deliberate hybrid that neither Guard nor Rose actually used: Guard's own system BRA includes propositional connectives $\sim, \supset$, modus ponens, and a Hilbert-style induction rule ([1, p. 10]; see Discussion), while Rose works with unary numerals rather than binary trees. The hybrid we formalize is strictly weaker than either author's setting — in particular it has no implication and no modus ponens — and the classical Hilbert–Bernays route to Gödel II is unavailable in it. The formalization is in Agda with `--safe --without-K --exact-split`. No postulates.

Theorem 1 (Gödel I, constructive form — `godelIToBot` in `ProofEAny.agda`)**.** *For any hypothesis equation H:*

$$H \vdash G \;\rightarrow\; H \vdash \mathbf{t} = \mathbf{f}$$

where G is the Gödel sentence.

Theorem 2 (Gödel II, constructive form — `conToBot` in `GodelIIFull.agda`)**.** *For any hypothesis equation H:*

$$H \vdash \mathrm{Con} \;\rightarrow\; H \vdash \mathbf{t} = \mathbf{f}.$$

Both results have standard corollaries: if H is consistent, then neither G nor Con is derivable from H.

2 The formal system

Trees

The ground data are finite binary trees:

$$a, b \;::=\; O \;\mid\; \langle a, b \rangle$$

where O is the leaf and $\langle a, b\rangle$ is the node. We define:

$$\mathbf{t} \;=\; O, \qquad \mathbf{f} \;=\; \langle O, O\rangle.$$

Here $\mathbf{t}$ represents equality (or "yes") and $\mathbf{f}$ represents inequality (or "no"). We write $\overline{n}$ for the unary encoding of $n \in \mathbb{N}$: $\overline{0} = O$, $\overline{n+1} = \langle O, \overline{n}\rangle$.

Terms and functors

Terms are built from trees, variables, and functor applications. We use t, u, v for terms, a, b for trees. The system has countably many variables $x_0, x_1, x_2, \ldots$. There are unary functors ($f, g : \mathrm{Fun}_1$) and binary functors ($h : \mathrm{Fun}_2$).

f, g	$::=$	I	identity
	$\mid$	π_1	first projection
	$\mid$	π_2	second projection
	$\mid$	$f \circ g$	composition
	$\mid$	$h[f, g]$	binary composition: $t \mapsto h(f(t), g(t))$
	$\mid$	$\mathrm{R}(z, s)$	tree recursion with base z and step s
	$\mid$	K_c	constant c
h	$::=$	P	pairing: $(t, u) \mapsto \langle t, u\rangle$
	$\mid$	C	left projection: $(t, u) \mapsto t$
	$\mid$	$\mathrm{Lf}(f)$	lift: $(t, u) \mapsto f(t)$
	$\mid$	$\mathrm{Ps}(f, h)$	post-compose: $(t, u) \mapsto f(h(t, u))$
	$\mid$	$\mathrm{Fn}(h_1, h_2, h)$	fan: $(t, u) \mapsto h(h_1(t, u),\ h_2(t, u))$
	$\mid$	If	conditional
	$\mid$	TreeEq	tree equality test

Terms:

$$t, u \;::=\; O \;\mid\; x_i \;\mid\; f(t) \;\mid\; h(t, u)$$

An *equation* is a pair $t = u$ of terms.

Embedding trees into terms

Every tree a determines a closed term $\hat{a}$:

$$\hat{O} = O, \qquad \widehat{\langle a, b\rangle} = \langle \hat{a}, \hat{b}\rangle.$$

(In Agda: `reify`.) Conversely, every closed term reduces to $\hat{a}$ for a unique a.

Derivation rules

A derivation is built from the rules below. Every rule may use a hypothesis H; we write $H \vdash t = u$.

Computation axioms.

$$
\begin{array}{ll}
\text{Identity:} & \mathrm{I}(t) = t \\
\text{First projection:} & \pi_1(\langle t, u\rangle) = t \\
\text{Second projection:} & \pi_2(\langle t, u\rangle) = u \\
\text{Constant:} & \mathrm{C}(t, u) = t \\
\text{Composition:} & (f \circ g)(t) = f(g(t)) \\
\text{Binary composition:} & h[f, g](t) = h(f(t),\ g(t)) \\
\text{Lift:} & \mathrm{Lf}(f)(t, u) = f(t) \\
\text{Post-compose:} & \mathrm{Ps}(f, h)(t, u) = f(h(t, u)) \\
\text{Fan:} & \mathrm{Fn}(h_1, h_2, h)(t, u) = h(h_1(t, u),\ h_2(t, u)) \\
\text{Constant functor:} & \mathrm{K}_c(t) = c
\end{array}
$$

Tree recursion.

$$
\begin{array}{ll}
\text{Base:} & \mathrm{R}(z, s)(O) = z \\
\text{Step:} & \mathrm{R}(z, s)(\langle t, u\rangle) = s\big(\langle t, u\rangle,\ \langle \mathrm{R}(z, s)(t), \mathrm{R}(z, s)(u)\rangle\big)
\end{array}
$$

Conditional.

$$
\begin{array}{ll}
\text{True branch:} & \mathrm{If}(\mathbf{t},\ \langle t, u\rangle) = t \\
\text{False branch:} & \mathrm{If}(\langle t_1, t_2\rangle,\ \langle u_1, u_2\rangle) = u_2
\end{array}
$$

More generally, If branches on whether its first argument is O (leaf) or a pair (node).

Tree equality.

$$
\begin{array}{ll}
\mathrm{TreeEq}(O, O) & = \mathbf{t} \\
\mathrm{TreeEq}(O, \langle t, u\rangle) & = \mathbf{f} \\
\mathrm{TreeEq}(\langle t, u\rangle, O) & = \mathbf{f} \\
\mathrm{TreeEq}(\langle t_1, t_2\rangle, \langle u_1, u_2\rangle) = & \mathrm{If}\big(\mathrm{TreeEq}(t_1, u_1),\ \langle \mathrm{TreeEq}(t_2, u_2), \mathbf{f}\rangle\big)
\end{array}
$$

TreeEq returns $\mathbf{t}$ for equal trees and $\mathbf{f}$ for unequal. The last clause says: if the left children differ, return $\mathbf{f}$ immediately; otherwise, recurse on the right children.

In particular $\mathrm{TreeEq}(t, t) = \mathbf{t}$ for all t, proved by **Schema F** (`treeEqSelf` in `GodelII.agda`).

Structural rules.

$$
\frac{}{t = t}\ (\mathrm{refl}) \qquad \frac{t = u}{u = t}\ (\mathrm{sym}) \qquad \frac{t = u \qquad u = v}{t = v}\ (\mathrm{trans})
$$

$$
\frac{t = u}{f(t) = f(u)}\ (\mathrm{cong}_1) \qquad \frac{t = u}{h(t, v) = h(u, v)}\ (\mathrm{cong}_L) \qquad \frac{t = u}{h(v, t) = h(v, u)}\ (\mathrm{cong}_R)
$$

Substitution.

$$
\frac{H \vdash t = u}{H \vdash t[c/x_i] = u[c/x_i]}\ (\mathrm{inst})
$$

Hypothesis.

$$
\frac{}{H \vdash H}\ (\mathrm{hyp})
$$

Schema F (tree induction). If f and g are unary functors, z a term, s a binary functor, and we have derivations of:

$$f(O) = z, \qquad f(\langle x_0, x_1\rangle) = s(\langle x_0, x_1\rangle,\ \langle f(x_0), f(x_1)\rangle),$$
$$g(O) = z, \qquad g(\langle x_0, x_1\rangle) = s(\langle x_0, x_1\rangle,\ \langle g(x_0), g(x_1)\rangle),$$

then $f(x_0) = g(x_0)$.

Schema F expresses the uniqueness of tree recursion: if f and g satisfy the same recurrence (same base z, same step s), they agree on all inputs. Restricted to unary trees $\overline{n}$, this gives ordinary mathematical induction.

Consistency

The system is *consistent* (relative to hypothesis H) if there is no derivation of $\mathbf{t} = \mathbf{f}$ from H.

3 Key components

3.1 Proof encoding (thFunTFor, Theorem 14)

thFunTFor is a unary functor (defined as $\mathrm{R}(O, \mathrm{thFunStep})$) that maps proof-encoding trees to the code of the equation they prove. The step function thFunStep is a 26-branch dispatch handling all derivation rules.

Every term t is encoded as a tree $\ulcorner t \urcorner$, and every equation $t = u$ as $\ulcorner t = u \urcorner = \langle \ulcorner t \urcorner, \ulcorner u \urcorner \rangle$. Every functor is similarly encoded.

Theorem 14 (thm14E)[1]: if $H \vdash t = u$, then there exist trees p_a, p_b such that:

1. $\mathrm{thFun}(\langle p_a, p_b\rangle) = \ulcorner t = u \urcorner$ (meta-level correctness),
2. $\vdash \mathrm{thFunTFor}(\langle \widehat{p_a}, \widehat{p_b}\rangle) = \widehat{\ulcorner t = u \urcorner}$ (object-level correctness, polymorphic in hypothesis).

The proof is by induction on the derivation, with 26 cases (1402 lines in `Thm14E.agda`).

Component (2) is *polymorphic*: the derivation uses only computation axioms and structural rules, never the hypothesis rule. This is crucial for Gödel II.

3.2 Coded substitution (substTFor)

substTFor is a unary functor implementing coded substitution: given a coded term $\ulcorner t \urcorner$, it replaces every occurrence of variable x_n by a replacement term r.

These two parameters (r and n) are represented by free variables in substTFor. To avoid collision with the variables x_0, x_1 used in **Schema F** and in the template equation, the formalization places them at high indices:

- x_{11} carries the replacement r,
- x_{12} carries the target index $\overline{n}$.

Closing these variables gives the fully applied version:

$$\mathrm{closedSubstTFor}(r, \overline{n}) \ = \ \mathrm{substTFor}[r/x_{11},\ \overline{n}/x_{12}].$$

Correctness: for any term t, $\mathrm{closedSubstTFor}(r, \overline{n})$ applied to $\widehat{\ulcorner t \urcorner}$ equals $\ulcorner \widehat{t[r/x_n]} \urcorner$.

[1] The name `thm14E` refers to Rose's Theorem 14 in [2], which is the D1-style reflection lemma. In Guard's lecture notes [1], "Theorem 14" is Gödel II itself (p. 16); the corresponding D1 lemma is Guard's Theorem 13 (p. 16, $f(x) = y \supset \mathrm{th}(D_f(x)) =$ "$fx{=}y$"). We follow Rose's numbering.

3.3 $\mathrm{TreeEq}(t,t) = \mathbf{t}$ via Schema F

$\mathrm{TreeEq}(t,t) = \mathbf{t}$ for all t is proved using **Schema F**. Define $f = \mathrm{TreeEq}[\mathrm{I},\mathrm{I}]$ (so $f(t) = \mathrm{TreeEq}(t,t)$) and $g = \mathrm{K}_{\mathbf{t}}$. Both satisfy the same recursion:

- f: by the computation axioms for TreeEq and the recursive structure of TreeEq on $\langle t_1, t_2\rangle$;
- g: by $\mathrm{K}_{\mathbf{t}}$; the step reduces to $\mathrm{If}(\mathbf{t}, \langle \mathbf{t}, \mathbf{f}\rangle) = \mathbf{t}$ since g returns $\mathbf{t}$ on all inputs.

Schema F gives $f(t) = g(t)$, i.e., $\mathrm{TreeEq}(t,t) = \mathbf{t}$.

4 The Gödel sentence and diagonal construction

Template

The Gödel sentence is built from a *template equation* with several free variables:

- x_0: will receive a proof encoding (the "proof slot");
- x_1: will receive a coded term (the "diagonal slot");
- x_{11}, x_{12}: the replacement and target parameters of substTFor (closed later).

The template is:

$$\mathrm{templateEqn} \;=\; \big(\mathrm{TreeEq}(\mathrm{thFunTFor}(x_0),\ \mathrm{substTFor}(x_1)) = \mathbf{f}\big).$$

It says: "the equation encoded by the proof in x_0 is not equal to the result of substituting via x_1." Leaving x_{11}, x_{12} free at this stage keeps the tree code of the template small.

Diagonalization

Let $N = \ulcorner\mathrm{templateEqn}\urcorner$ be the tree encoding of the template. The Gödel sentence is obtained by substituting $\hat{N}$ (the numeral for N) into the diagonal slot:

$$G \;=\; \mathrm{templateEqn}[\hat{N}/x_1].$$

After this substitution, G says (informally): "for any proof tree in x_0, the equation it encodes is not the result of substituting N into the template" — i.e., "no proof tree proves G."

Abstract definitions (cached)

The following are computed once in `SubstTForPrecomp.agda` and cached:

- N: tree code of the template
- $\hat{N}$: the numeral $\widehat{N}$
- $G = \mathrm{templateEqn}[\hat{N}/x_1]$
- $\ulcorner G\urcorner$: tree encoding of G
- $\mathrm{cstf} = \mathrm{closedSubstTFor}(\widehat{\hat{N}},\ \overline{1})$: the substitution functor closed with replacement = code of the template numeral and target = variable x_1

Heavy computation (one-time) and its elimination

In the original chain, `SubstTForPrecomp.agda` contains one step requiring ∼108 seconds (computed once, cached in `.agdai`): proving by `refl` that after instantiating x_{11} and x_{12}, the open substTFor becomes cstf. This requires Agda to traverse $\widehat{\widehat{N}}$ (∼400K term nodes) with substitution acting as identity on the closed subterm.

The mathematical argument does not require this traversal. A general "substitution is identity on a reify-image" lemma, together with a schematic version of the substTFor→cstf closing equation, eliminates it. We provide both: see the *Fast chain* below.

Fast chain (no 108-second computation)

A parallel chain of files, all suffixed `Fast`, replaces the `instForm2 = refl` call with a schematic proof built from four lemmas (in `Guard/Nelson/`):

- substReify : $\mathrm{subst}\, x\, r\, (\widehat{t}) = \widehat{t}$, by induction on the tree t;
- substTForClose : $\mathrm{substF}_1\, x_{11}\, (\widehat{r})\, (\mathrm{substF}_1\, x_{12}\, (\widehat{n})\, \mathrm{substTFor}) = \mathrm{closedSubstTFor}(\widehat{r}, \widehat{n})$, schematic in r, n;
- thFunTForSubstEq: the same closing equation propagated through the 26-branch dispatch chain of thFunTFor, so that the embedded substTFor in case 23 becomes closedSubstTFor;
- instFormGen: a schematic bridge that, after ruleInst on x_{12} then x_{11}, converts the open template equation into the closed one.

The crucial point is that all four lemmas are universally quantified over the replacement and target trees: *no specific tree (in particular, not $\widehat{\widehat{N}}$) ever appears in their types or proofs.* Agda type-checks them in constant time independently of the size of $\widehat{\widehat{N}}$.

The fast chain reorders the deployment of ruleInst in the Gödel I derivation: instead of substituting x_{11}, x_{12} on the equation that already contains cstf (which forces unfolding $\widehat{\widehat{N}}$), it substitutes on the *open* equation that contains thFunTFor and substTFor schematically, and only afterwards bridges to the closed form via the lemmas above. This corresponds exactly to Rose's order of operations: substitute the parameters into the open template first, then specialize.

Connection with Simmons (1988). The four schematic Nelson helpers are an instance of the general principle described by Simmons [5]: an *indicial reduction*. In Simmons' setting, for each functional A on a base Ξ there is a primitive recursive *indicial function* α_A such that $A(\underline{l}; \underline{u}(x); \{\underline{d}\}) = \{\alpha_A(\underline{l}; \underline{u}; \underline{d})\}(x)$: the pointwise evaluation of A at x is replaced by an arithmetic computation α_A on codes followed by a single catalogue lookup at the end. Simmons' Proposition 5.1 states that α_A exists for every Ξ-functional and is "constructed by mimicking the construction of the parent functional." His Theorem 6.1 then shows that any recursion defined by such functionals is itself in the catalogue, by running an arithmetic recursion on the codes via the indicial functions and applying the lookup only at the end.

The Fast chain follows the same pattern in the Guard/Rose equational setting:

- substReify is the indicial reduction of $\mathrm{subst}\, x\, r\, (\widehat{t})$, proved by induction on the tree t.
- substTForClose is the indicial reduction of double substitution into substTFor, proved by mimicking the construction of substTFor.

- thFunTForSubstEq is the indicial reduction of double substitution into thFunTFor, proved by mimicking the 23-branch dispatch chain of thFunTFor (Simmons' Proposition 5.1 in action).
- instFormGen packages the indicial reductions of the open template equation into a single bridge step (Simmons' Theorem 6.1 in action).

The 108-second `instForm2 = refl` is precisely the "fundamental obstruction" Simmons' Section 1 warns about: it asks Agda to evaluate a recursion on functor expressions pointwise, which forces materialization of $\widehat{\hat{N}}$ (a value with ∼400K tree nodes). The Fast chain replaces this pointwise evaluation by Simmons-style indicial reduction, in which no specific tree (in particular, not $\widehat{\hat{N}}$) ever appears in the type or proof of any of the helpers, and the catalogue lookup is performed only once at the very end. Simmons' principle is from 1988 and was originally aimed at clarifying classical primitive recursion; it transports cleanly to the setting of an Agda formalization where "pointwise evaluation" is replaced by "Agda's definitional unfolding."

Files. The fast chain consists of, in dependency order: `SubstTForPrecompFast.agda`, `SubstTForPrecomp2Fast.a` `GodelIIFast.agda`, `ProofEAnyFast.agda`, and `GodelIIFullFast.agda`. Each file is a copy of the corresponding original with the substitution-bridge restructured to use the schematic lemmas; the original files are preserved unchanged.

Timings (clean rebuild). Both numbers below measure type-checking from `.agda` source after deleting the `.agdai` files for the diverging tail of the dependency tree. The shared upstream files (`Term`, `Step`, `Thm14E`, etc.) are already cached.

Chain	**Clean rebuild time**
Original (via `SubstTForPrecomp` + `instForm2 = refl`)	∼103 s
Fast (via `SubstTForPrecompFast` + Nelson schematic helpers)	∼7 s

A ∼15× speedup, with no postulates and the same final theorems (`conToBot`, `godelII`). The original chain remains in the repository for reference, since `instForm2 = refl` is a striking demonstration of the cost of letting Agda reduce a closed $\widehat{\hat{N}}$ unnecessarily.

5 Proof of Gödel I

Theorem 3 (Gödel I, constructive form)**.** *For any hypothesis equation H: $H \vdash G \;\to\; H \vdash \mathbf{t} = \mathbf{f}$.*

Corollary 4 (Gödel I, standard form — `godelI` in `ProofEAny.agda`)**.** *Consistent(H) $\;\to\;$ $H \vdash G \;\to\; \bot$.*

Both forms remove the ProofE(H) assumption from the original proof. The key enabler is mkProofEAny, which constructs a proof encoding for any equation using the trans-from-two-refls technique.

The proof in `GodelII.agda` (0.2 seconds, 168 lines) follows Rose's Theorem 18 [2]. We assume $H \vdash G$ and derive $H \vdash \mathbf{t} = \mathbf{f}$:

1. **Assume** $H \vdash G$.
2. **Theorem 14** gives a proof encoding e with

$$H \vdash \text{ thFunTFor}(e) = \widehat{\ulcorner G \urcorner}.$$

3. **diagFTarget** (precomputed): $H \vdash \ \mathrm{cstf}(\hat{N}) = \widehat{\ulcorner G \urcorner}$.

4. **Chain** steps 2 and 3 through $\widehat{\ulcorner G \urcorner}$ as middle term (trans + sym):

$$H \vdash \ \mathrm{thFunTFor}(e) = \mathrm{cstf}(\hat{N}).$$

   The term $\widehat{\ulcorner G \urcorner}$ appears only as the middle of a transitivity chain — it is never touched by inst.

5. **Instantiate** G with e (inst on x_0):

$$H \vdash \ \mathrm{TreeEq}(\mathrm{thFunTFor}(e),\ \mathrm{substTFor}(\hat{N})) = \mathbf{f}.$$

6. **Replace** $\mathrm{thFunTFor}(e)$ with $\mathrm{cstf}(\hat{N})$ using cong_L and the chain from step 4:

$$H \vdash \ \mathrm{TreeEq}(\mathrm{cstf}(\hat{N}),\ \mathrm{substTFor}(\hat{N})) = \mathbf{f}.$$

7. **Instantiate** x_{11} and x_{12} (inst, using the cached computation from Section 4): the open $\mathrm{substTFor}(\hat{N})$ becomes $\mathrm{cstf}(\hat{N})$:

$$H \vdash \ \mathrm{TreeEq}(\mathrm{cstf}(\hat{N}),\ \mathrm{cstf}(\hat{N})) = \mathbf{f}.$$

8. **Reflexivity** ($\mathrm{TreeEq}(t, t) = \mathbf{t}$ via **Schema F**):

$$H \vdash \ \mathrm{TreeEq}(\mathrm{cstf}(\hat{N}),\ \mathrm{cstf}(\hat{N})) = \mathbf{t}.$$

9. **Contradiction**: by trans, $H \vdash \ \mathbf{t} = \mathbf{f}$, violating consistency.

6 Proof of Gödel II

6.1 The internal consistency statement

We define the (open) consistency statement as the equation in one free variable x:

$$\mathrm{Con}(x) \ \equiv \ \mathrm{TreeEq}\big(\mathrm{thFunTFor}(x),\ \ulcorner \mathbf{t} = \mathbf{f} \urcorner\big) = \mathbf{f}.$$

Thus $\mathrm{Con}(x)$ states: *the proof encoded by x is not a proof of* $\mathbf{t} = \mathbf{f}$. Since the system has no quantifiers, consistency is expressed as this open equation; the intended reading is "for all x, $\mathrm{Con}(x)$". This formulation is predicative: no internal universal quantification or derivability conditions are required.

6.2 A hypothesis-polymorphic reflection lemma (DC1-poly)

The key technical invariant is the following strengthening of Theorem 14.

Lemma 5 (DC1-poly)**.** *There is a primitive-recursive encoding* enc *that maps every derivation* $d : H \vdash t = u$ *to a closed term* $\mathrm{enc}(d)$ *for which*

$$\vdash \mathrm{thFunTFor}(\mathrm{enc}(d)) = \ulcorner t = u \urcorner.$$

The absence of any ambient hypothesis on the right-hand side is the content of the lemma: the derivation is *absolute*, using no hypothesis rule. The hypothesis H of the input derivation is absorbed into the *structure* of $\mathrm{enc}(d)$, not into the ambient context of the output equation.

Proof. Theorem 14 is the operation enc: by recursion on d it produces a closed tree $\mathrm{enc}(d)$ together with a derivation of $\mathrm{thFunTFor}(\mathrm{enc}(d)) = \ulcorner t = u \urcorner$. Inspecting `thm14E`, each of its 26 cases emits its part of the correctness derivation using only computation axioms and structural rules; none of them use the hypothesis rule. The resulting derivation is therefore absolute.

Uses of the hypothesis rule in d (i.e. `ruleHyp` nodes, where d concludes $H \vdash H$) are encoded by splicing in the reflexivity-based witness of H provided by `mkProofEAny`, a closed tree T_H for which $\vdash \mathrm{thFunTFor}(T_H) = \ulcorner H \urcorner$ is derivable by computation axioms alone. This absorbs the hypothesis into $\mathrm{enc}(d)$'s *shape*, so that the correctness equation itself remains hypothesis-free. □

Remark 6. *Because* $\mathrm{enc}(d)$ *is closed and* thFunTFor *is a fixed term, the equation* $\vdash \mathrm{thFunTFor}(\mathrm{enc}(d)) = \ulcorner t = u \urcorner$ *is a* ground computational identity*: one derivation of it serves in every subsequent proof context. Note also that* thFunTFor *is a conclusion-extractor, not a judgment-encoder: the equation records only the conclusion of* d*, not the antecedent* H*.*

6.3 Diagonal collapse

We isolate the final step common to Gödel I and Gödel II.

Lemma 7 (Diagonal collapse)**.** *For any closed term* c*, if* $H \vdash \mathrm{TreeEq}(c, c) = \mathbf{f}$*, then* $H \vdash \mathbf{t} = \mathbf{f}$*.*

Proof. By **Schema F**, $\mathrm{TreeEq}(c, c) = \mathbf{t}$ for every c. Combine with the assumption by transitivity. □

6.4 Statement

Theorem 8 (Gödel II, constructive form — `conToBot` in `GodelIIFull.agda`)**.** *For any hypothesis equation* H*,*

$$H \vdash \mathrm{Con}(x) \implies H \vdash \mathbf{t} = \mathbf{f}.$$

Corollary 9 (Gödel II, standard form — `godelII` in `GodelIIFull.agda`)**.** *If* H *is consistent, then* $H \nvdash \mathrm{Con}(x)$*.*

6.5 Proof

The proof has three steps.

Step 1: A concrete contradiction derivation. By Gödel I (Theorem 3), taking the hypothesis $H_0 = G$, we obtain a derivation

$$d_\perp \ : \ G \vdash \mathbf{t} = \mathbf{f}.$$

This derivation is produced constructively using `ruleHyp` (to obtain $G \vdash G$) followed by the Gödel I argument; it involves no consistency assumption.

Step 2: Reflect the derivation absolutely. Applying Lemma 5 to $d_\bot$ yields a closed term $enc_\bot = \mathrm{enc}(d_\bot)$ with

$$\vdash \mathrm{thFunTFor}(enc_\bot) = \ulcorner \mathbf{t} = \mathbf{f} \urcorner.$$

This is an absolute derivation; in particular it holds under the ambient hypothesis H under which $\mathrm{Con}(x)$ is given. (The construction of enc on a `ruleHyp` node uses the reflexivity-based witness of G produced by `mkProofEAny`, which is how G is absorbed into $enc_\bot$ at every hypothesis use-site in $d_\bot$.)

Step 3: Instantiate consistency and collapse. Assume $H \vdash \mathrm{Con}(x)$. By substitution (`ruleInst`) with $x := enc_\bot$,

$$H \vdash \mathrm{TreeEq}\big(\mathrm{thFunTFor}(enc_\bot),\ \ulcorner \mathbf{t} = \mathbf{f} \urcorner\big) = \mathbf{f}.$$

Rewriting with the equality from Step 2 and congruence,

$$H \vdash \mathrm{TreeEq}\big(\ulcorner \mathbf{t} = \mathbf{f} \urcorner,\ \ulcorner \mathbf{t} = \mathbf{f} \urcorner\big) = \mathbf{f}.$$

Applying Lemma 7 (diagonal collapse) gives $H \vdash \mathbf{t} = \mathbf{f}$. □

6.6 Discussion

The proof uses only a single instance of reflection (DC1), strengthened to hypothesis-polymorphism. No internal modus ponens (D3) or iterated reflection (D2) is required.

Conceptually, Gödel II reduces to

- constructing one explicit derivation of contradiction under the hypothesis G (Gödel I);
- reflecting it to a closed proof code $enc_\bot$;
- instantiating consistency at this witness;
- collapsing via $\mathrm{TreeEq}(c, c) = \mathbf{t}$.

Thus Gödel II is obtained by transporting a *single concrete proof witness* across hypotheses, rather than by reasoning about all proofs internally.

Integration with earlier sections. The diagonal construction (template, N, cstf, etc.) remains entirely in Section 5 (Gödel I), where it belongs. No reference to `SubstTForPrecomp`, `instForm2`, or the fast chain is needed here; those concern the construction of $d_\bot$, not its use. The modal presentation below is a direct abstraction of Lemmas 5 and 7.

6.7 Modal presentation (`Prov.agda`, `GodelIIProv.agda`)

Lemmas 5 and 7 admit a direct modal abstraction, in the style of Dijk and Gietelink-Oldenziel [6] (who gave Joyal's arithmetic-universe proof of Gödel's theorems). We make this reading explicit in two additional files, leaving `GodelIIFull.agda` untouched.

Definition 10 (`Prov.agda`)**.** *For an equation e, define the provability predicate*

$$\mathrm{Prov}(e) \ = \ \big\{\, t : \mathrm{Term} \ \big|\ \mathrm{thFunTFor}(t) = \widehat{\ulcorner e \urcorner} \,\big\}$$

(the record has fields `enc` *and* `corr`*, and the correctness witness is polymorphic in the hypothesis).*

Three operators, each a few lines of Agda, suffice:

- **Necessitation.** $\mathit{ProofE}(H) \to (H \vdash e) \to \mathrm{Prov}(e)$, obtained by packaging the output of `thm14E`. This is Lemma 5 in record form, and corresponds to Lemma 5.15 of [6].
- **Internal Cantor diagonal.** Exactly Lemma 7; this is the shared tail of Rose's Theorems 17 and 18 and of Theorems 5.19 and 5.20 of [6], each of which reduces at the final step to the same collapse of a diagonal equalizer.
- **Gödel II as a one-liner.** Let $\mathit{provBot} : \mathrm{Prov}(\mathbf{t} = \mathbf{f})$ be the necessitation of the Gödel I derivation. Then `conToBot` is literally:

```
conToBot dCon =
  diagContradict codeBotT
    (ruleTrans (ruleSym (congL TreeEq codeBotT (corr provBot)))
               (ruleInst zero (enc provBot) dCon))
```

  i.e. "instantiate Con at the witness of $\mathrm{Prov}(\bot)$, rewrite by its correctness, apply the diagonal collapse".

The refactor does not shorten the total development — `Thm14E.agda` is unchanged — but it *renames* the load-bearing step into the form it has in the categorical proof: the Cantor-diagonal equalizer, applied once at the level of Prov. Gödel I and Gödel II are then visibly two instances of the same collapse, on $\widehat{cGS}$ and on $\widehat{\ulcorner\bot\urcorner}$ respectively.

7 Discussion

Comparison with predecessors

The reflection-and-diagonal architecture for Gödel's incompleteness theorems in recursive arithmetic is due to Rose and Guard in the early 1960s, and several attempts at Gödel II for Rose-style or Guard-style systems exist in the literature. We summarize how each one handles the load-bearing internalization step.

Guard 1963 (BRA, Theorem 14). Guard [1] proves Gödel II (his Theorem 14, p. 16) inside his system BRA. It is important to note that BRA is *not* a purely equational system: its primitive symbols include propositional connectives $\sim$ and $\supset$, axioms 11–13 are standard Hilbert-style propositional axioms, and the rules of inference include modus ponens and a Hilbert-style induction rule "from $P(0)$ and $P(x) \supset P(sx)$ infer $P(x)$" ([1, p. 10]). Guard's proof of Gödel II is a classical Hilbert–Bernays internalization: his Theorem 13 ($f(x) = y \supset \mathrm{th}(D_f(x))$ = "$fx{=}y$", p. 16) is the D1 reflection lemma, and he then constructs explicit functors f, g, h (p. 17) encoding a modus-ponens chain that takes an alleged proof of the Gödel sentence to a proof of "0=1". This argument is sound *because* BRA has implication and modus ponens at the object level; it cannot be ported to a purely equational system.

Rose 1961 (Theorem 18). Rose [2] works in a purely equational system (closer to ours). He proves Gödel II (his Theorem 18) by applying his Theorem 16 — an *internalization of Schema F* — to a function built from the diagonal construction. Cook [3] (p. 89, footnote) points out a possible error in Theorem 16 (p. 134 of [2]): the function $f(s(x))$ is neither identically zero nor identically

non-zero, which invalidates the dichotomy on which the internal Schema F argument depends. This is, to our knowledge, the only attempt in the literature to do Gödel II purely equationally, and the gap is exactly at the point where the proof would have to leave the equational discipline to perform internal case analysis.

Ryan 1978. Ryan [4] reformulates Rose's program for general recursive equational arithmetic and states the corresponding versions of his Lemma 1 (the D1 reflection lemma), Lemma 3 (Gödel I), and the Main Theorem (Gödel II). He explicitly remarks that "the proofs of the lemmas and Main Theorem of this section parallel the proofs of the corresponding results in Rose" [4], and supplies no replacement argument for the step that Cook flagged. Ryan's note is therefore evidence of *transmission* of Rose's program to a new system, not evidence of an independent repair of the gap.

Cook 1975 (PV, Theorem 4.4). Cook [3] also defines a Rose-style equational system PV, and proves Gödel II for PV as his Theorem 4.4. However, Cook explicitly says (p. 90) that he "will actually work in the system PV1, since this is easier." PV1 is PV extended with propositional connectives $\sim, \supset, \&, \vee, \equiv$, axioms E1–E7, the truth-functional inference rule IMP, and an induction-on-notation rule ([3, Section 3]). Cook's Lemma 4.5 ($\vdash_{\mathrm{PV1}} f(x_1, \ldots, x_k) = y \supset \mathrm{PROOF}(\mathrm{FORM}_f(\bar{x}, y), \mathrm{GEN}_f(\bar{x}, y)) = 1$) is then a Hilbert–Bernays D1 statement, and his Lemma 4.7 is an explicit D3-style internal modus ponens combining a coded proof witness with an actual proof witness via a CONTRA-construction. The result is transferred back to PV via Cook's Theorem 3.10 (an equation is provable in PV iff it is provable in PV1). Cook's strategy is therefore: *when the equational system does not suffice, escape to a propositional extension in which Hilbert–Bernays goes through, and then transfer the final equational consequence back.* Cook's argument is presented as an outline; the load-bearing Lemma 4.5 is asserted via his Theorem 2.12 rather than constructed.

Summary. The four prior treatments fall into two camps. Guard 1963 and Cook 1975 add propositional logic to the system in order to run a Hilbert–Bernays-style internalization (D1 + internal modus ponens). Rose 1961 and Ryan 1978 stay equational but rely on internalized Schema F (Rose Theorem 16), which Cook flagged as gappy. We are not aware of any prior treatment that completes a Gödel II proof for a purely equational system without either escaping to propositional logic or invoking Rose's Theorem 16.

Our approach

The system formalized here is purely equational with **Schema F** as its only induction principle: no propositional connectives, no modus ponens, no Hilbert-style induction. This is strictly weaker than BRA, PV1, or any of the classical settings. In particular the standard Hilbert–Bernays strategy is unavailable in principle: there is no $\supset$ to internalize and no MP to chain. Our proof of Gödel II (`conToBot` in `GodelIIFull.agda`) avoids both the Cook escape route and Rose's Theorem 16 by a different deployment of the same D1-style reflection lemma (`thm14E`) that Rose, Ryan, Cook, and Guard all use:

1. we run the Gödel I proof under the hypothesis G itself (using ruleHyp), obtaining a concrete derivation $d_\bot : G \vdash \mathbf{t} = \mathbf{f}$;

2. we apply `thm14E` to $d_\bot$, obtaining a closed proof code $enc_\bot$ together with a derivation $corrBot : \mathrm{thFunTFor}(enc_\bot) = \widehat{\ulcorner \bot \urcorner}$;

3. we observe that *corrBot* is *hypothesis-polymorphic* — it uses only computation axioms and structural rules, never the hypothesis rule — so it holds in any hypothesis context, in particular under the assumption $H \vdash \mathrm{Con}$;

4. we instantiate Con at $enc_\perp$, rewrite $\mathrm{thFunTFor}(enc_\perp)$ to $\widehat{\ulcorner \perp \urcorner}$ via *corrBot*, and contradict the result via $\mathrm{TreeEq}(t, t) = \mathbf{t}$.

The technical move that makes this work is the **hypothesis-polymorphism of `thm14E`'s correctness derivation**. In Hilbert–Bernays language, we use only D1; the iterated reflection D2 and the internal modus ponens D3 are not needed, because we transport a single concrete proof code at the meta-level rather than constructing a universal-over-proofs argument inside the system.

The role of Theorem 14

The entire non-trivial content of the proof resides in `thm14E`. The 26 case files verify that thFunTFor correctly tracks each derivation rule. The Gödel II proof applies `thm14E` to the specific derivation $d_\perp$ (the Gödel I argument with hypothesis G), obtaining an encoding whose thFunTFor image is the contradiction code. The instantiation of Con at this encoding then produces the contradiction.

Positioning

To our knowledge:

- This is the first complete proof of Gödel II for a purely equational system that neither escapes to a propositional extension (as Cook does) nor relies on Rose's Theorem 16.
- This is the first machine-checked proof of Gödel II in a Rose/Guard-style equational setting. Prior mechanizations of Gödel II that we are aware of work in systems with quantifiers and propositional connectives (hereditarily finite set theory, abstract derivability conditions, PA).
- The deployment of D1's hypothesis-polymorphism as the load-bearing technical ingredient is, as far as we have been able to determine, not isolated as a named property in earlier proofs of Gödel II.

The mathematical core (reflection lemma + diagonal) is Rose's; the deployment of that core via single-witness hypothesis-polymorphic transport, and the formalization in a system too weak to express the classical proof, are new.

Cook's "programming approach"

Cook [3, Section 8] suggests that a natural formalization of feasibly constructive proofs should take a *programming approach*, where "proving $f(x) = g(x)$ amounts to proving the equivalence of two programs." The system formalized here realizes this vision literally: proving $f(x_0) = g(x_0)$ via **Schema F** *is* proving that two recursive programs (the functors f and g) compute the same function. Every derivation in the system is a chain of equational rewrites on functor expressions — no quantifiers, no propositional connectives, only computation axioms and the uniqueness-of-recursion principle. Cook himself did not formalize Gödel II in this discipline; his Section 4 proof relies on the propositional extension PV1. In this sense the present work supplies the "programming-approach" Gödel II that Cook's framing suggests but did not realize.

8 Correspondence with predecessors

Rose 1961

Rose [2]	**This formalization**
$th(y)$	thFunTFor
$se(x, y, z)$ / $st(x, y, z)$	substTFor / closedSubstTFor
$nu(x)$	implicit via $\hat{N}$
N (template code)	templateCode
Schema F	ruleF (uniqueness of recursion)
Theorem 10	thm14E (ProofE)
Theorem 14 (D1 reflection)	thm14E
Theorems 15–16	Bypassed (see Discussion); cf. [3] on Thm 16
Theorem 17	diagFTarget
Theorem 18 (Gödel II)	`godelII` in `GodelII.agda` (= Gödel I form)
—	`godelII` in `GodelIIFull.agda` (= Gödel II)

Guard 1963 and Cook 1975

Concept	**Guard 1963 / Cook 1975**	**This formalization**
Numerals	unary $0, s(0), \ldots$ / dyadic strings	binary trees $O, \langle a, b\rangle$
Propositional logic	present ($\sim, \supset$, MP)	*absent*
Induction principle	$P(0), P(x) \supset P(sx) / P(x)$	Schema F (equational)
Proof enumerator	th(y) / PROOF(m, n)	thFunTFor
D1 reflection	Guard Th 13 / Cook Lemma 4.5	thm14E
Internal MP (D3)	Guard's f, g, h functors / Cook's CONTRA	*not used*
Internal consistency	th(y) $\neq$ "0=1" / PROOF([0=1], y) = 0	Con
Gödel II	Guard Th 14 / Cook Th 4.4 (in PV1)	`conToBot`
Strategy	internalize Gödel I via D1+D3	meta-construct $enc_\perp$, transport via polymo

The naming clash on "Theorem 14" is worth flagging: in Guard 1963, Theorem 14 is Gödel II itself; in Rose 1961, Theorem 14 is the D1 reflection lemma. The Agda file `Thm14E.agda` is named after Rose's numbering (D1), not Guard's.

9 File structure

File	Time	Role
Base.agda	< 1s	Natural numbers, equality, sigma types
Term.agda	< 1s	Terms, functors, coding functions
Step.agda	< 1s	Derivation system (26 rules + **Schema F**)
StepReduce.agda	< 1s	Derived reduction lemmas
ThFun.agda	< 1s	Metalevel proof evaluator
ThFunCorrect.agda	< 1s	Metalevel correctness
ThFunTFor.agda	< 1s	Object-level proof evaluator (26 branches)
ThFunTForDefs.agda	< 1s	Definitions for thFunTFor cases
ThFunTForCases0–3.agda	< 1s	Per-tag case proofs
ThFunTForCorrectDefs.agda	< 1s	Correctness infrastructure
Thm14E.agda	< 1s	Theorem 14 (1402 lines, all 26 cases)
SubstTFor.agda	< 1s	Coded substitution functor
SubstCorrect.agda	< 1s	Substitution correctness
SubstTForCorrect.agda	< 1s	closedSubstTFor correctness
SubstTForPrecomp.agda	108s	**Heavy**: precomputed abstractions
SubstTForPrecomp2.agda	< 1s	Assembly from opaque pieces
NdDispatchProofs.agda	< 1s	Node-dispatch case proofs
PairPassthrough.agda	< 1s	Pair passthrough lemmas
PairPassthroughAll.agda	< 1s	Extended passthrough
IntermediatePassthrough.agda	< 1s	Intermediate passthrough
ExtractorRed.agda	< 1s	Extractor reduction lemmas
GodelII.agda	< 1s	Gödel I (168 lines) + treeEqSelf
ProofEAny.agda	< 1s	mkProofEAny + strengthened Gödel I
GodelIIFull.agda	< 1s	**Gödel II** ($\sim$90 lines)
Prov.agda	< 1s	Provability modality, necessitation, diagContradict
GodelIIProv.agda	< 1s	Gödel II, modal presentation via `Prov`
CookPV.agda	< 1s	Boolean ops, De Morgan, excluded middle
NoPChecker.agda	< 1s	No proof-checker (corollary of Gödel II)
Fast chain (parallel, no 108-second computation):		
Nelson/SubstReify.agda	< 1s	substReify, substClosedSTF
Nelson/SubstTForClose.agda	< 1s	substTForClose (v12-first)
Nelson/ThFunTForSubst.agda	< 1s	thFunTForSubstEq + thFunTForClosed
Nelson/InstForm.agda	< 1s	instFormGen (schematic bridge)
SubstTForPrecompFast.agda	< 1s	As `SubstTForPrecomp`, no `instForm2`
SubstTForPrecomp2Fast.agda	< 1s	As `SubstTForPrecomp2`
GodelIIFast.agda	< 1s	Gödel I via schematic bridge
ProofEAnyFast.agda	< 1s	`mkProofEAny` + fast strengthened Gödel I
GodelIIFullFast.agda	< 1s	**Gödel II**, fast variant

All results compile with Agda 2.9.0 using `--safe --without-K --exact-split`. No postulates. The original chain takes $\sim$103 s on a clean rebuild (dominated by `instForm2 = refl`); the parallel fast chain takes $\sim$7 s on a clean rebuild and proves the same theorems.

References

[1] J. R. Guard, *Lecture Notes on Recursive Arithmetic*, Princeton University, 1963.

[2] H. E. Rose, "On the consistency and undecidability of recursive arithmetic," *Zeitschr. f. math. Logik und Grundlagen d. Math.*, vol. 7, pp. 124–135, 1961.

[3] S. A. Cook, "Feasibly constructive proofs and the propositional calculus," *Proceedings of the Seventh Annual ACM Symposium on Theory of Computing*, pp. 83–97, 1975.

[4] W. Ryan, "Gödel's second incompleteness theorem for general recursive arithmetic," *Zeitschr. f. math. Logik und Grundlagen d. Math.*, vol. 24, pp. 457–459, 1978.

[5] H. Simmons, "The realm of primitive recursion," *Archive for Mathematical Logic*, vol. 27, pp. 177–188, 1988.

[6] J. van Dijk and A. Gietelink-Oldenziel, "Gödel's Incompleteness after Joyal," arXiv:2004.10482, 2020.